

Complete Java Web Development Handbook

1. Introduction to Java Web Development

Java Web Development is used to build dynamic web applications.

A typical Java web application contains:

- Frontend (JSP / HTML / CSS)
 - Backend Logic (Servlets)
 - Database Connectivity (JDBC)
 - Architecture (MVC)
 - Build Management (Maven)
-

2. MVC Architecture (Most Important)

Before learning JDBC, Servlet, or JSP, students must understand MVC.

What is MVC?

MVC stands for:

- Model
- View
- Controller

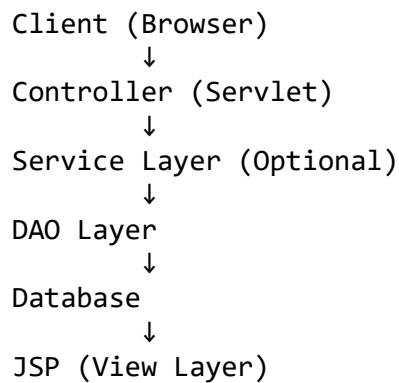
It separates:

- UI
- Business Logic
- Database Logic

This makes applications:

- Clean
 - Maintainable
 - Scalable
 - Industry standard
-

MVC Flow Diagram



Responsibilities

Layer	Responsibility
JSP	UI / Display
Servlet	Request handling
DAO	Database operations
DTO	Data transfer
Database	Data storage

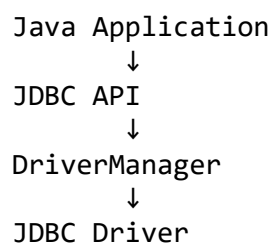
3. JDBC (Java Database Connectivity)

What is JDBC?

JDBC is used to connect Java applications with relational databases like:

- MySQL
 - Oracle
 - PostgreSQL
-

JDBC Architecture



↓
Database

JDBC Flow

1. Load Driver
 2. Create Connection
 3. Create PreparedStatement
 4. Execute Query
 5. Process ResultSet
 6. Close Connection
-

Basic JDBC Example

```
Class.forName("com.mysql.cj.jdbc.Driver");

Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb",
    "root",
    "root"
);

PreparedStatement ps =
    con.prepareStatement("SELECT * FROM users");

ResultSet rs = ps.executeQuery();

while(rs.next()) {
    System.out.println(rs.getString("username"));
}

con.close();
```

Statement Types

1. Statement

Used for static SQL queries.

```
Statement stmt = con.createStatement();
```

Problem:

- Vulnerable to SQL Injection
-

2. PreparedStatement (Recommended)

Used for parameterized queries.

```
PreparedStatement ps =  
con.prepareStatement("SELECT * FROM users WHERE id=?");
```

Advantages:

- Prevents SQL Injection
 - Faster
 - Reusable
-

3. CallableStatement

Used to execute stored procedures.

```
CallableStatement cs =  
con.prepareCall("{call MY_PROCEDURE(?)}");
```

Common JDBC Errors

Error	Reason
No suitable driver	Driver not loaded
Access denied	Wrong username/password
SQL syntax error	Invalid SQL query
NullPointerException	Object not initialized
Connection timeout	DB not running

Adding MySQL Connector WITHOUT Maven Steps

1. Download MySQL Connector JAR

Download:

mysql-connector-j.jar

2. Create Java Project

File → New → Java Project

3. Add JAR to Classpath

Right Click Project

→ Build Path

→ Add External JARs

→ Select mysql connector jar

4. Servlets

What is Servlet?

Servlet is a Java class used to:

- Receive HTTP requests
 - Process requests
 - Send HTTP responses
-

Servlet Lifecycle

init()

↓

service()

↓

destroy()

init()

Called only once when servlet loads.

service()

Handles all requests.

Internally calls:

- doGet()
 - doPost()
-

destroy()

Called before servlet is removed.

Basic Servlet Example

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    protected void doGet(
        HttpServletRequest req,
        HttpServletResponse res
    ) throws IOException {

        res.getWriter().println("Hello Servlet");
    }
}
```

doGet vs doPost

doGet	doPost
Used for fetching data	Used for sending data
Parameters visible in URL	Parameters hidden
Less secure	More secure

RequestDispatcher

Used to forward request from servlet to JSP.

```
RequestDispatcher rd =
request.getRequestDispatcher("home.jsp");

rd.forward(request, response);
```

sendRedirect

```
response.sendRedirect("home.jsp");
```

Forward vs Redirect

forward()	sendRedirect()
Same request	New request
Faster	Slower
Request data available	Request data lost
URL unchanged	URL changes

Session Handling

Create Session

```
HttpSession session = request.getSession();  
  
session.setAttribute("user", "admin");
```

Get Session Value

```
HttpSession session = request.getSession(false);  
  
String user =  
(String) session.getAttribute("user");
```

Invalidate Session

```
HttpSession session = request.getSession(false);  
  
if(session != null) {  
    session.invalidate();  
}
```

Sending Data to JSP

```
request.setAttribute("error",  
    "Invalid Credentials");
```

```
RequestDispatcher rd =  
request.getRequestDispatcher("login.jsp");  
  
rd.forward(request, response);
```

5. JSP (Java Server Pages)

What is JSP?

JSP is used to create dynamic web pages.

It combines:

- HTML
 - Java
-

JSP Lifecycle

```
JSP  
↓  
Servlet  
↓  
Compiled Class  
↓  
Response
```

JSP Elements

Element	Syntax
Scriptlet	<% %>
Expression	<%= %>
Declaration	<%! %>

Scriptlet Example

```
<%  
int a = 10;  
out.println(a);  
%>
```

Expression Example

```
<%= 10 + 20 %>
```

Declaration Example

```
<%!  
int square(int x) {  
    return x * x;  
}  
%>
```

JSP Directives

Directive	Purpose
page	Page configuration
include	Include file
taglib	JSTL usage

Include Directive

```
<%@ include file="header.jsp" %>
```

Why Scriptlets Are Avoided

Modern applications avoid Java code in JSP.

Instead use:

- EL
 - JSTL
-

6. Expression Language (EL)

What is EL?

EL is used to access data without Java code.

EL Syntax

```
${name}  
${param.username}  
${sessionScope.user}
```

EL Scopes

Scope	Description
pageScope	Current page
requestScope	Current request
sessionScope	Session data
applicationScope	Entire application

EL Operators

```
${10 + 20}  
${age > 18}  
${empty user}  
${not empty user}
```

Ternary Operator

```
${age > 18 ? "Adult" : "Minor"}
```

Working with Objects

```
${student.name}  
${student.email}
```

Working with List

```
${list[0]}
```

7. JSTL (Java Standard Tag Library)

What is JSTL?

JSTL removes Java code from JSP.

Taglib Declaration

```
<%@ taglib
uri="http://java.sun.com/jsp/jstl/core"
prefix="c" %>
```

Important JSTL Tags

c:if

```
<c:if test="${age > 18}">
    Adult
</c:if>
```

c:forEach

```
<c:forEach var="i"
begin="1" end="5">

    ${i}

</c:forEach>
```

c:set

```
<c:set var="name"
value="Guru"/>
```

c:choose

<c:choose>

```
<c:when test="${marks > 50}">
    Pass
</c:when>

<c:otherwise>
    Fail
</c:otherwise>
```

</c:choose>

Why JSTL?

Advantages:

- Cleaner JSP
 - No Java code
 - Better readability
 - Easier maintenance
-

8. DAO and DTO

DTO (Data Transfer Object)

DTO carries data between layers.

```
BookDTO book = new BookDTO();
book.setTitle("Java");
```

DAO (Data Access Object)

DAO handles database operations.

```
dao.saveBook(book);
```

DAO + DTO Flow

Servlet
↓
DTO Object
↓
DAO Layer
↓
Database

9. Maven Web Project Setup

Steps to Create Maven Web Project

1. Create Maven Project

File → New → Maven Project

2. Select Packaging = war

WAR is used for web applications.

3. Add Dependencies

Servlet API

```
<dependency>  
  <groupId>javax.servlet</groupId>  
  <artifactId>javax.servlet-api</artifactId>  
  <version>4.0.1</version>  
  <scope>provided</scope>  
</dependency>
```

JSTL

```
<dependency>  
  <groupId>javax.servlet</groupId>  
  <artifactId>jstl</artifactId>  
  <version>1.2</version>  
</dependency>
```

MySQL Connector

```
<dependency>  
  <groupId>com.mysql</groupId>  
  <artifactId>mysql-connector-j</artifactId>  
  <version>9.4.0</version>  
</dependency>
```

4. Configure Tomcat

Window → Preferences
→ Server → Runtime Environment
→ Add Apache Tomcat

5. Add JSP and Servlet Files

- Servlet in:
src/main/java

- JSP in:
src/main/webapp

10. Mini Project Flow

Login Flow

```
login.jsp  
  ↓  
LoginServlet  
  ↓  
DAO  
  ↓  
Database  
  ↓  
home.jsp
```

Book Store Flow

```
add-book.jsp  
  ↓  
BookServlet
```

↓
BookDAO
↓
Database

11. Best Practices

- Use PreparedStatement instead of Statement
 - Avoid Java code in JSP
 - Use EL and JSTL
 - Follow MVC architecture
 - Separate DAO and DTO properly
 - Close DB connections properly
 - Keep JSP only for UI
-

12. Common Student Mistakes

Mistake	Solution
Wrong DB URL	Check database name
JSTL not working	Add JSTL dependency
404 Servlet error	Check URL mapping
NullPointerException	Initialize objects properly
Session data missing	Use forward instead of redirect

13. Assignments

Assignment 1

Build Login System.

Assignment 2

Create Student CRUD.

Assignment 3

Create Book Store Form.

Assignment 4

Implement Session Handling.

Assignment 5

Display List using JSTL.
